Proyecto Grupal - Gestión Bibliotecaria

Programación I (17047)

Profesor: Abel Israel Laime Huanca

Grupo 7 - Integrantes:
Pelardas, Pablo Martín - 1225156
Miño, Sofía Micaela - 1224140
Micheloni, Valentina - 1191607
Marturell Moyegas, Jean Carlos - 1183701
Correa, Elizabeth Florencia Solana - 1221915

Repositorio del proyecto

https://github.com/pablopelardas/GestionBibliotecaria

Índice

Índice 1
Equipo de trabajo 3
Introducción 3
Objetivo del Proyecto 3
Alcance 4
Tecnología Utilizada 4
Conceptos Académicos Aplicados 4
Etapa 1 - Versión Preliminar 5
1. Estructura del proyecto 5
2. Decisiones de diseño 5
2.1 Estructura de Datos de Libros 5
2.2 Índice de ISBN para Búsqueda Binaria 6
2.3 Integración con Préstamos 7
3. Funcionalidades Implementadas 8
3.1 Menú Principal 8
3.2 Módulo Usuarios 9
3.2.1 Función: agregar_usuario(datos) 9
3.2.2 Función: modificar_usuario(user_id, datos) 9
3.2.3 Función: eliminar_usuario(user_id) 9
3.2.4 Función: listar_usuarios() 10
3.2.5 Función: obtener_usuario_completo(user_id) 10
3.3 Módulo Libros 10
3.3.1 Función: agregar_libro(datos) 11
3.3.2 Función: modificar_libro(libro_id, nuevos_datos) 11
3.3.3 Función: eliminar_libro(libro_id) 11
3.3.4 Función: listar_libros() 12
3.4 Módulo Préstamos 12
3.4.1 Función: registrar_prestamo (libro_id, user_id) 12
3.4.2 Función: registrar_devolución (gener, libro_id) 13
3.4.3 Función: obtener_prestamos_activos() 13
3.4.4 Integración con la Capa de Presentación 13
3.5 Módulo Buscador 14
3.5.1 Búsqueda Binaria Recursiva por ISBN 14
3.5.2 Búsqueda recursiva por texto libre 14
3.5.3 Búsqueda por Género 17
3.6 Módulo Reportes 17
3.6.1 Reporte de totales 17
3.6.2 Más prestados 18
3.7 Utilidades 19
3.7.1 Sistema de paginación genérico 19
3.7.1 Utilidades de consola 20
4. Uso de Listas y Cadenas 21

4.1 Listas..... 21
4.2 Cadenas..... 22
5. Funciones Principales Esbozadas..... 23
5.1 Usuarios..... 23
5.2 Libros..... 23
5.3 Préstamos..... 23
5.4 Buscador (implementado)..... 24
5.5 Reportes..... 24
6. Datos de prueba..... 25
6.1 Usuarios..... 25
6.2 Libros..... 25
6.3 Préstamos..... 25
7. Conclusiones..... 26
7.1 Logros principales..... 26
7.2 Desafíos superados..... 26
7.3 Conceptos académicos consolidados..... 27
7.4 Oportunidades de mejora..... 27
7.5 Reflexión Final..... 27

Equipo de trabajo

Nombre	Rol Principal	Módulo Asignado	Responsabilidades
Correa Elizabeth Florencia Solana	Líder técnica	Usuarios	Implementación lógica de negocio, alta, baja, modificación y listado de usuarios
Miño Sofía Micaela	Backend	Gestión de libros	Implementación alta, baja y modificación de libros
Micheloni Valentina	Documentación	Préstamos y devoluciones de libros	Implementación registro y devolución de préstamos, historial
Pelardas Pablo Martín	QA/Test	Buscador	Implementación búsquedas (índice, texto libre, género), sistema de paginación
Marturell Jean	Presentación	Reportes Avanzados	Implementación reportes estadísticos y análisis de datos

Introducción

Las bibliotecas, tanto públicas como privadas, enfrentan desafíos en la gestión manual de sus recursos. El control de préstamos, inventario de libros, registro de usuarios y generación de reportes suelen realizarse mediante procesos manuales o sistemas obsoletos que resultan ineficientes.

Este proyecto surge como respuesta a la necesidad de modernizar y agilizar la gestión bibliotecaria, proporcionando una herramienta digital que simplifique las operaciones diarias y mejore la experiencia tanto de los bibliotecarios como de los usuarios.

Objetivo del Proyecto

Desarrollar un Sistema de Gestión Bibliotecaria completo y funcional que permita:

- Gestionar el inventario de libros: alta, baja, modificación y búsqueda eficiente de ejemplares.
- Administrar usuarios: registro, actualización y control de lectores

- Controlar préstamos: registro de préstamos, devoluciones y seguimiento de libros en circulación.
- Generar reportes: estadísticas y análisis de uso de la biblioteca.
- Facilitar búsquedas: implementación de algoritmos eficientes para localizar libros rápidamente.

Alcance

El sistema está diseñado para:

- Bibliotecas pequeñas a medianas (hasta 10,000 ejemplares)
- Operación local (sin requerimientos de red)
- Interfaz de consola intuitiva y fácil de usar
- Persistencia de datos mediante archivos JSON
- Procesamiento eficiente mediante algoritmos optimizados (búsqueda binaria, recursividad)

Tecnología Utilizada

- Lenguaje: Python 3
- Almacenamiento: Archivos JSON organizados en estructura de directorios.
- Arquitectura: en capas (Presentación, Lógica de Negocio, Datos)

Conceptos Académicos Aplicados

- Estructuras de datos: listas, matrices, diccionarios, cadenas
- Algoritmos de búsqueda: búsqueda binaria, búsqueda secuencial
- Recursividad: recorrido de directorios, búsqueda binaria
- Funciones: modularización, parámetros, valores de retorno
- Manejo de archivos: lectura y escritura de JSON
- Validación de datos: control de entradas del usuario

Etapa 1 - Versión Preliminar

1. Estructura del proyecto

El proyecto sigue una arquitectura de tres capas:

📂 data/ Contiene la base de datos de la aplicación

- 📂 libros/ Fichas de libros clasificadas por género (.JSON)
- 📂 usuarios/ Fichas de usuarios (.JSON)
- 📂 préstamos/ Historial de préstamos (.JSON)

📂 modelos/ Clases: Libro, Usuario, Préstamo (.PY)
📂 presentación/ Menús de consola (.PY)
📂 negocio/ Servicios: libro_service.py, usuario_service.py, prestamo_service.py, reporte_service.py
📂 utils/ Utilidades y helpers (.PY)
main.py Entrada principal del programa (.PY)

2. Decisiones de diseño

2.1 Estructura de Datos de Libros

Decisión: Usar estructura de directorios por género en lugar de un único archivo JSON.
Justificación:

- Facilita la organización y búsqueda por género.
- Permite búsqueda recursiva en directorios. (requisito académico)
- Mejora el rendimiento al no cargar todos los libros en memoria.
- Simula un sistema de archivos real más escalable.

Formato de archivo de libro (data/libros/{género}/{libro id}.json):

```
data > libros > biografia > {} 1e8bb33f-1ad8-4a16-9b31-680f25f13e7e.json > ...
{
    "libro_id": "1e8bb33f-1ad8-4a16-9b31-680f25f13e7e",
    "isbn": "978-5-3282-9052-7",
    "title": "Pressure event one responsibility",
    "autor": "Pamela Carter",
    "genero": "biografia",
    "disponible": false,
    "prestamo_actual": "3F71A62D078B4EE4B635935A22",
    "historial_prestamos": [
        {
            "prestamo_numero": 26,
            "user_id": "3F71A62D078B4EE4B635935A22",
            "regresado": false
        }
    ]
}      feat: nueva estructura libros, archivos individual…
```

El ISBN identifica la edición del libro, mientras que el libro_id identifica cada ejemplar físico. Múltiples ejemplares pueden compartir el mismo ISBN.

2.2 Índice de ISBN para Búsqueda Binaria

Decisión: Mantener un índice ordenado de ISBNs en /data/libros/isbn_index.json
Justificación:

- Permite implementar búsqueda binaria recursiva.
- El índice se mantiene ordenado para aprovechar el algoritmo.
- Vincula ISBNs con las ubicaciones de todos sus ejemplares.

Formato del índice

```
{
    "isbn": "9780225310157",
    "title": "Next",
    "autor": "Brandi Lee",
    "genero": "ciencia",
    "ejemplares": [
        {
            "libro_id": "18850242-d81f-43c5-8033-76dcc4e514ce",
            "ruta": "data/libros/ciencia/18850242-d81f-43c5-8033-76dcc4e514ce.json"
        }
    ]
},
{
    "isbn": "9780228723805",
    "title": "Authority member win",
    "autor": "Benjamin Fernandez",
    "genero": "arte",
    "ejemplares": [
        {
            "libro_id": "aa6d9e97-15c4-497b-9f17-67dde0043e03",
            "ruta": "data/libros/arte/aa6d9e97-15c4-497b-9f17-67dde0043e03.json"
        }
    ]
}
```

2.3 Integración con Préstamos

Decisión: Los datos de préstamos se mantienen en data/prestamos/prestamos.json pero se integran en la visualización de libros.

Justificación:

- Separación de responsabilidades (cada módulo gestiona sus datos).
- Facilita la actualización independiente de cada módulo.
- Permite acceder fácilmente a la información del cliente o libro prestado en las búsquedas.

3. Funcionalidades Implementadas

3.1 Menú Principal

El sistema cuenta con un menú jerárquico con las siguientes opciones:

```
═══════════════════════════════════════
    SISTEMA DE GESTIÓN BIBLIOTECARIA
═══════════════════════════════════════
1. Gestión de Usuarios
2. Gestión de Libros
3. Gestión de Préstamos
4. Buscador
5. Reportes Avanzados
6. Salir
```

Características del menú:

- Validación de opciones ingresadas
- Limpieza de consola entre operaciones
- Submenús para cada módulo
- Navegación intuitiva

Archivo: presentation/menu.py

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

==============================
   SISTEMA DE BIBLIOTECA
==============================
1. Gestión de Usuarios
2. Gestión de Libros
3. Préstamos y Devoluciones
4. Reportes y Búsquedas
0. Salir
==============================
Seleccione una opción: 1

--- Gestión de Usuarios ---
1. Alta de usuario
2. Modificar usuario
3. Eliminar usuario
4. Listar usuarios
0. Volver al menú principal
Seleccione una opción: 1
👉 Aquí iría la lógica de alta de usuario

--- Gestión de Usuarios ---
1. Alta de usuario
2. Modificar usuario
3. Eliminar usuario
4. Listar usuarios
0. Volver al menú principal
Seleccione una opción: 2
👉 Aquí iría la lógica de modificar usuario
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

   SISTEMA DE BIBLIOTECA
==============================
1. Gestión de Usuarios
2. Gestión de Libros
3. Préstamos y Devoluciones
4. Reportes y Búsquedas
0. Salir
==============================
Seleccione una opción: 4

--- Reportes y Búsquedas ---
1. Libros más prestados
2. Usuarios con más préstamos
3. Búsqueda por ISBN
4. Búsqueda por texto libre
0. Volver al menú principal
Seleccione una opción: 0

==============================
   SISTEMA DE BIBLIOTECA
==============================
1. Gestión de Usuarios
2. Gestión de Libros
3. Préstamos y Devoluciones
4. Reportes y Búsquedas
0. Salir
==============================
Seleccione una opción: 0
¡Hasta pronto!
```

3.2 Módulo Usuarios

Permite realizar el **ABML (Alta, Baja, Modificación y Listado)** de los usuarios del sistema. Cada usuario se guarda como un archivo `.json` dentro de la carpeta `data/usuarios/`. Cada usuario se identifica mediante un **ID único (UUID)**.

3.2.1 Función: agregar_usuario(datos)

Parámetros:

- `datos` (dict): contiene los campos `'nombre'`.

Salida:

- Retorna `True` si el usuario fue agregado correctamente, `False` si ocurrió un error

Características:

- Crea automáticamente la carpeta `data/usuarios/` si no existe.
- Genera un identificador único (UUID) para cada usuario.
- Inicializa los campos `'activo'` (True) y `'historial_prestamos'` (lista vacía).
- Guarda la información del usuario en formato JSON con el nombre `<uuid>.json`.

3.2.2 Función: modificar_usuario(user_id, datos)

Parámetros:

- `user_id` (str): ID del usuario a modificar.
- `datos` (dict): contiene los nuevos valores a actualizar.

Salida:

- Retorna `True` si el usuario fue modificado correctamente, `False` si no se encontró.

Características:

- Busca el archivo JSON correspondiente al `user_id`.
- Mantiene los valores originales si el usuario deja un campo vacío.
- Permite actualizar información como nombre, apellido, DNI o email.

3.2.3 Función: eliminar_usuario(user_id)

Parámetros:

- `user_id` (str): ID del usuario a eliminar.

Salida:

- Retorna `True` si el usuario fue eliminado correctamente, `False` si no se encontró.

Características:

- Busca y elimina el archivo JSON correspondiente.

3.2.4 Función: listar_usuarios()

Salida:

- Devuelve una lista con todos los usuarios registrados, incluyendo su nombre, apellido, DNI y estado (activo/inactivo).

Características:

- Recorre la carpeta `data/usuarios/`.
- Retorna una lista vacía si no hay usuarios registrados.
- Se integra con la capa de presentación (`presentation/usuarios.py`) para mostrar el listado por consola.

3.2.5 Función: obtener_usuario_completo(user_id)

Parámetros:

- `user_id` (str): ID del usuario a consultar.

Salida:

- Devuelve un diccionario con toda la información del usuario.

Características:

- Busca y lee el archivo JSON correspondiente.
- Permite obtener todos los datos asociados al usuario, útil para mostrar detalles o validar préstamos.

3.3 Módulo Libros

Permite realizar el **ABML (Alta, Baja, Modificación y Listado)** de los libros del sistema.
Cada libro se guarda como un archivo .json dentro de una carpeta correspondiente a su género, ubicada en data/libros/.

Cada libro se identifica mediante un **ID único (UUID)**, mientras que los ISBN se usan para identificar la edición de una obra.

3.3.1 Función: agregar_libro(datos)

Parámetros:

- datos (dict): contiene los campos 'isbn', 'title', 'autor' y 'genero'

Salida:

- Retorna el libro_id del ejemplar creado

Características:

- Verifica la existencia del libro mediante el isbn
- Crea automáticamente la carpeta del género si no existe
- Agrega el libro al índice_isbn o, en caso de ya existir, agrega el ejemplar al correspondiente libro
- Genera un identificador único (UUID) para cada ejemplar de libro
- Inicializa los campos ‘disponible’, ‘prestamo_actual’ e ‘historial_prestamos’
- Guarda la información del libro en formato JSON
- Fuerza la reconstrucción del índice

3.3.2 Función: modificar_libro(libro_id, nuevos_datos)

Parámetros:

- libro_id (str): ID del libro a modificar
- nuevos_datos (dict): datos a actualizar

Salida:

- Retorna True si se modificó correctamente, False si no se encontró

Características:

- Recorre todas las carpetas de géneros para encontrar el archivo
- Mantiene los valores originales si el usuario deja un campo vacío (ej, si quiere modificar sólo el título)
- Modifica tanto el archivo JSON del correspondiente libro como el archivo índice_isbn

3.3.3 Función: eliminar_libro(libro_id)

Parámetros:

- libro_id (str): ID del libro a eliminar

Salida:

- Retorna True si el libro fue eliminado correctamente, False si no se encontró

Características:

- Busca el libro en todas las carpetas de géneros
- Elimina el archivo JSON correspondiente y el ejemplar en índice_isbn
- Fuerza la reconstrucción del índice

3.3.4 Función: listar_libros()

Salida:

- Devuelve una lista con todos los libros registrados, incluyendo su título, autor y género

Características:

- Recorre todas las carpetas de géneros
- Retorna una lista vacía si no hay libros registrados
- Se integra con la capa de presentación (presentation/libros.py) para mostrar el listado por consola

3.4 Módulo Préstamos

Permite registrar, devolver y listar los préstamos de libros realizados por los usuarios. Cada préstamo se almacena en el archivo data/prestamos/prestamos.json, manteniendo un registro completo del historial de operaciones realizadas en el sistema.
Además, el módulo actualiza automáticamente el estado de disponibilidad de los libros al momento de registrar un préstamos o una devolución, garantizando la integridad del inventario.

3.4.1 Función: registrar_prestamo (libro_id, user_id)

Parámetros:

- genero (str): Carpeta del género del libro (ruta del ejemplar)
- libro_is (str): ID del libro a prestar
- user_id (str): ID del usuario que solicita el préstamo

Salida:

- Retorna True si el préstamo se registró correctamente
- Retorna False si el libro no existe o ya se encuentra prestado

Características:

- Verifica que el libro esté disponible antes de registrar el préstamo.
- Marca el campo disponible del libro como False.
- Crea un nuevo registro en data/prestamos/prestamos.json con los siguientes datos:
    - ➔ libro_id
    - ➔ user_id
    - ➔ fecha_prestamos
    - ➔ devuelvo (inicializado en False)
- Asigna automáticamente la fecha del préstamo utilizando la fecha y hora actual

- Se integra con la capa de presentación (presentación/prestamos.py) para la interacción mediante consola.

3.4.2 Función: registrar_devolución (gener, libro_id)

Parámetros:
- libro_id (str): ID del libro a devolver.

Salida:
- Retorna True si la devolución se registró correctamente.
- Retorna False si no se encuentra un préstamo activo para ese libro.

Características:
- Busca en el archivo data/prestamos/prestamos.json el préstamos activo asociado al libro.
- Marca el campo devuelto como True y agrega la fecha_devolución actual.
- Actualiza el archivo del libro en data/libro/{género}/{id}.json, marcando el campo disponible como True.
- Garantiza que un mismo libro no pueda tener más de un préstamo activo al mismo tiempo.

3.4.3 Función: obtener_prestamos_activos()

Salida:
- Retorna el archivo data/prestamos/prestamos.json y filtra los préstamos en los cuales el campo devuelto sea False.
- Devuelve una lista de diccionarios con información del préstamo:
  - ➔ libto_id, user_id, fecha_prestamo y genero.
- Se utiliza desde la capa de presentación para mostrar los préstamos vigentes en consola.

3.4.4 Integración con la Capa de Presentación

El archivo presentation/prestamos.py permite al usuario interactuar con el módulo de préstamos a través de un menú de consola.
Las opciones disponibles son:
- Registrar préstamo
- Registrar devolución
- Listar préstamos vigentes

Cada opción ejecuta una de las funciones del servicio (préstamos_service.py), mostrando mensajes de confirmación o error según el resultado.
Esta integración permite que el sistema mantenga sincronizados los estados de los libros y los registros de préstamos.

3.5 Módulo Buscador

3.5.1 Búsqueda Binaria Recursiva por ISBN

Función: busqueda_binaria_isbn(isbn_buscado)
Salida: muestra todos los ejemplares que comparten ese ISBN con su estado de disponibilidad.

```
=== Búsqueda por ISBN ===
Ingrese el ISBN a buscar: 978-2-2583-6118-3

✓ Libro encontrado:
  ISBN: 978-2-2583-6118-3
  Título: Letter business audience
  Autor: Beverly Jones
  Género: tecnologia
  Total ejemplares: 1

  Disponibilidad:
    Disponibles: 1
    Prestados: 0

¿Desea ver detalles de los ejemplares? (s/n): s

  Ejemplar 1:
    ID: 75476a51-33d7-41f2-bb98-f15b4aead681
    Disponible: Sí
    Préstamos históricos: 0

Presione Enter para continuar...
```

3.5.2 Búsqueda recursiva por texto libre

Función: búsqueda_recursiva_texto(texto_busqueda)
Salida: muestra todos las ediciones de libros (ISBN único) que contengan la cadena buscada en el título, autor o isbn.
Características:

- Es case insensitive
- Coincidencias parciales
- Paginación automática de 15 resultados por página

```
python3 main.py

=== Búsqueda por Texto Libre ===
Busca en título, autor o ISBN
Ingrese el texto a buscar: And

Buscando 'And'...

✓ Se encontraron 96 coincidencias

¿Desea ver todos los resultados? (s/n):
```

```
   Género: ciencia
   Estado: ✓ Disponible

3. Model religious
   Autor: Randall Smith
   ISBN: 978-7-5908-1382-0
   Género: ciencia
   Estado: ✓ Disponible

4. Half total
   Autor: Michael Sandoval
   ISBN: 978-4-8465-3060-4
   Género: ciencia
   Estado: ✓ Disponible

5. Beat the level
   Autor: Stephanie Hernandez
   ISBN: 978-7-3040-5191-1
   Género: ciencia
   Estado: ✓ Disponible

6. International finally exactly
   Autor: Andrew Page
   ISBN: 978-4-7223-3002-3
   Género: ciencia
   Estado: ✗ Prestado

7. Someone model community
   Autor: Jacob Anderson
   ISBN: 978-2-9151-2906-7
   Género: ciencia
   Estado: ✓ Disponible

8. Trouble job
   Autor: Andrea Taylor
   ISBN: 978-9-3521-9203-5
   Género: ciencia
   Estado: ✗ Prestado

9. Next
   Autor: Brandi Lee
   ISBN: 978-0-2253-1015-7
   Género: ciencia
   Estado: ✗ Prestado

10. Husband artist
   Autor: Anthony Dawson
   ISBN: 978-8-9491-7703-0
   Género: ciencia
   Estado: ✓ Disponible

------------------------------------------------------------
Opciones:
  [S] Página siguiente
  [0] Volver

Seleccione una opción:
```

3.5.3 Búsqueda por Género

Función: buscar_por_genero(genero)
Salida: muestra todos las ediciones de libros (ISBN único) de un mismo genero.
Características:

- Muestra todos los ejemplares de un género específico
- Muestra estadísticas: total de ejemplares, disponibles y estados
- Paginación de resultados

```
=== Búsqueda por Género ===

Géneros disponibles:
  1. Arte
  2. Biografia
  3. Ciencia
  4. Ficcion
  5. Filosofia
  6. Historia
  7. Otros
  8. Tecnologia

Seleccione un género (número): 1

Buscando libros de 'arte'...

✓ Se encontraron 123 libros de arte:

Estadísticas:
  Total: 123
  Disponibles: 81
  Prestados: 42

¿Desea ver la lista de libros? (s/n):
```

3.6 Módulo Reportes

3.6.1 Reporte de totales

- **Función:**report_totals(export=False)

```python
# 1. Totals
def report_totals(self, export=False):
    total_users = len(self.users)
    total_books = len(self.books)
    active_loans = sum(1 for loan in self.loans.values() if not loan.get("regresado", False))

    report = {
        "Total de Usuarios": total_users,
        "Total de Libros": total_books,
        "Préstamos Activos": active_loans,
    }

    self._print_report("Totales de la Biblioteca", report)

    if export:
        self._export_to_csv("report_totals.csv", report)
```

- **Salida:**

Diccionario con totales: Total de Usuarios, Total de Libros, Préstamos Activos. Además, imprime el reporte por consola.

- **Características:**

Cuenta usuarios, libros y préstamos activos. export=True: guarda CSV en reports/report_totals.csv.

```
=== Totales de la Biblioteca ===
Total de Usuarios: 300
Total de Libros: 1000
Préstamos Activos: 387
CSV report saved as C:\Users\User\OneDrive\Documentos\UADE\02. Programacion 1\TP\GestionBibliotecaria\reports\report_totals.csv
```

3.6.2 Más prestados

· **Función:** report_most_borrowed_books(top_n=5, export=False, only_active=False)

```python
def report_most_borrowed_books(self, top_n=5, export=False, only_active=False):
    # Si only_active=True, contar solo préstamos no devueltos
    borrowed_books = [
        loan.get("libro_id")
        for loan in self.loans.values()
        if loan.get("libro_id") and (not only_active or not loan.get("regresado", False))
    ]
    counter = Counter(borrowed_books)

    report = []
    for book_id, count in counter.most_common(top_n):
        libro = self.books.get(book_id, {})
        report.append({
            "Libro": libro.get("title", "Desconocido"),
            "Autor": libro.get("autor", "Desconocido"),
            "Veces Prestado": count
        })

    self._print_table("Libros Más Prestados", report)

    if export:
        self._export_list_to_csv("report_most_borrowed_books.csv", report)

    return report
```

- **Salida:** Lista (top N) de diccionarios: { "Libro": ..., "Autor": ..., "Veces Prestado": ... }.
- **Características:** Calcula frecuencia por libro_id a partir de loans. only_active=True cuenta solo préstamos no devueltos. export=True: guarda CSV en reports/report_most_borrowed_books.csv.

```
=== Libros Más Prestados ===
Libro | Autor | Veces Prestado
----------------------------------------
Body item poor | Kathryn Baker | 1
Year notice image ball | Steven Robbins | 1
True different | Christopher Wilkins | 1
CSV report saved as C:\Users\User\OneDrive\Documentos\UADE\02. Programacion 1\TP\GestionBibliotecaria\reports\report_most_borrowed_books.csv
```

3.6.3 Top Users

- **Función:** report_top_users(top_n=5, export=False)

```python
def report_top_users(self, top_n=5, export=False):
    # Contar préstamos por usuario (histórico)
    user_counts = {}
    for loan in self.loans.values():
        user_id = loan.get("user_id")
        if not user_id:
            continue
        user_counts[user_id] = user_counts.get(user_id, 0) + 1

    sorted_users = sorted(user_counts.items(), key=lambda x: x[1], reverse=True)

    report = []
    for uid, count in sorted_users[:top_n]:
        usuario = self.users.get(uid, {})
        libros_actuales = len(usuario.get("libros_prestados", []))
        report.append({
            "Usuario": usuario.get("nombre", "Desconocido"),
            "Préstamos Totales": count,
            "Libros Actuales": libros_actuales
        })

    self._print_table("Usuarios con Más Préstamos", report, headers=["Usuario", "Préstamos Totales", "Libros Actuales"]

    if export:
        self._export_list_to_csv("report_top_users.csv", report)

    return report
```

- **Salida:**

Lista de diccionarios: { "Usuario": ..., "Préstamos Totales": ..., "Libros Actuales": ... }.

- **Características:**

Cuenta préstamos por user_id (histórico) usando loans. Libros actuales provienen de users[user_id]["libros_prestados"]. **export=True:** guarda CSV en reports/report_top_users.csv.

```
=== Usuarios con Más Préstamos ===
Usuario | Préstamos Totales | Libros Actuales
----------------------------------------
Charles Durham | 8 | 8
Body item poor | Kathryn Baker | 1
Year notice image ball | Steven Robbins | 1
True different | Christopher Wilkins | 1
```

3.7 Utilidades

3.7.1 Sistema de paginación genérico

Función: mostrar_resultados_paginados(resultados, funcion_mostrar_item, titulo, por_pagina)
Salida: muestra la cantidad por_pagina de items en la lista de resultados utilizando la callback funcion_mostrar_item y con el título que recibe por parámetro.
Características:

- Funciona con cualquier tipo de dato
- Recibe una callback para mostrar cada item
- Navegación: siguiente (S), anterior (A), ir a página (número), salir (0)

Archivo: utils/paginación.py

```
    Género: arte
    Estado: ✓ Disponible

14. Service example
    Autor: Christina Bond
    ISBN: 978-4-9367-1991-5
    Género: arte
    Estado: ✗ Prestado

15. Financial his
    Autor: Valerie Garza
    ISBN: 978-4-6483-8475-2
    Género: arte
    Estado: ✓ Disponible

--------------------------------------------
Opciones:
  [S] Página siguiente
  [1-9] Ir a página específica
  [0] Volver

Seleccione una opción:
```

3.7.1 Utilidades de consola

Funciones:

- limpiar_consola(): limpia la pantalla según el sistema operativo
- pausar(): espera el input del usuario

Archivo: utils/consola.py

4. Uso de Listas y Cadenas

4.1 Listas

Ejemplos de uso en código:

1. Lista de resultados de búsqueda:

```
    resultados = []
    for archivo in genero_dir.glob('*.json'):
        with open(archivo, 'r', encoding='utf-8') as f:
            libro = json.load(f)
            resultados.append(libro)

    return resultados
```

2. Índice de ISBNs (lista de diccionarios)

```
    """        feat: búsqueda libros por isbn, género o texto lib…, Pablo Pelar
    global _indice_isbn_cache

    if _indice_isbn_cache is None:
        archivo_indice = obtener_directorio_libros() / 'indice_isbn.json'
        with open(archivo_indice, 'r', encoding='utf-8') as f:
            _indice_isbn_cache = json.load(f)

    return _indice_isbn_cache
```

3. Lista de géneros

```
def listar_generos():
    """
    Lista todos los géneros disponibles.

    Returns:
        list: Lista de nombres de géneros orde
    """
    dir_libros = obtener_directorio_libros()
    generos = []

    for item in dir_libros.iterdir():
        if item.is_dir():
            generos.append(item.name)

    return sorted(generos)
```

4. Slicing de listas para paginación

```python
# Calcular rango de resultados para esta página
inicio = pagina_actual * por_pagina
fin = min(inicio + por_pagina, len(resultados))

# Mostrar encabezado
print(f"\n{'='*60}")
print(f"{titulo}")
print(f"{'='*60}")
print(f"Página {pagina_actual + 1} de {total_paginas}")
print(f"Mostrando registros {inicio + 1} al {fin} de {len(resultados)} total")
print(f"{'='*60}")

# Mostrar resultados de la página actual
for i, item in enumerate(resultados[inicio:fin], inicio + 1):
    funcion_mostrar_item(item, i)
    print()  # Línea en blanco entre items

# Mostrar opciones de navegación
```

4.2 Cadenas

1. Búsqueda case-insensitive con .lower()

```python
# Buscar en título, autor o ISBN
if (texto_busqueda in libro.get('title', '').lower() or
    texto_busqueda in libro.get('autor', '').lower() or
    texto_busqueda in libro.get('isbn', '').lower()):

    resultados.append(libro)
```

2. Formateo de strings con f-strings

```python
if resultado:
    print(f"\n✓ Libro encontrado:")
    print(f"  ISBN: {resultado['isbn']}")
    print(f"  Título: {resultado['title']}")
    print(f"  Autor: {resultado['autor']}")
    print(f"  Género: {resultado['genero']}")
    print(f"  Total ejemplares: {len(resultado['ejemplares'])}")

    # Mostrar estado de ejemplares
```

3. Validación de cadenas con .strip()

```python
# Obtener opción del usuario
opcion = input("\nSeleccione una opción: ").strip().lower()
```

5. Funciones Principales Esbozadas

5.1 Usuarios

Función	Parámetros	Retorno	Estado
obtener_nombre_usuario(user_id)	user_id:str	str	✅
cargar_usuarios()	-	dict	✅
agregar_usuario(datos)	datos: dict	bool	✅
modificar_usuario(user_id, datos)	user_id: str, datos: dict	bool	✅
eliminar_usuario(user_id)	user_id: str	bool	✅

5.2 Libros

Función	Parámetros	Retorno	Estado
agregar_libro(datos)	datos: dict	str (libro_id)	✅
modificar_libro(libro_id, nuevos_datos)	libro_id: str, datos: dict	bool	✅
eliminar_libro(libro_id)	libro_id: str	bool	✅
listar_libros()	-	dict(libros)	✅

5.3 Préstamos

Función	Parámetros	Retorno	Estado
registrar_prestamo(libro_id,user_id)	libro_id: str, user_id: str	bool	✅
registrar_devolucion(libro_id)	libro_id: str	bool	✅
obtener_prestamos_usuario(user_id)	user_id: str	list[dict]	✅
obtener_prestamos_activos()	-	list[disct]	✅

5.4 Buscador (implementado)

Función	Parámetros	Retorno	Estado
busqueda_binaria_sibn(isbn)	isbn: str	dict o None	✅
busqueda_recursiva_texto(texto)	texto: str	list[dict]	✅
buscar_por_genero(genero)	genero: str	list[dict]	✅
listar_generos()	-	list[str]	✅

5.5 Reportes

Función	Parámetros	Retorno	Estado
generar_reporte_general()	-	str	✅
generar_reporte_prestamos()	-	str	✅
generar_reporte_usuarios_morosos()	-	list[dict]	✅
exportar_reporte(nombre, formato)	nombre: str, formato: str	bool	✅

6. Datos de prueba

6.1 Usuarios

- Total: 20 usuarios registrados
- Archivo: data/usuarios/usuarios.json
- Campos:
    - user_id
    - user
        - nombre
        - libros_prestados

6.2 Libros

- Total: 1000 ejemplares
- 8 géneros
- Campos
    - libro_id
    - isbn
    - title
    - autor
    - genero
    - disponible
    - prestamo_actual
    - historial_prestamos

6.3 Préstamos

- Total: 500+
- Activos: ~400
- Archivo: data/prestamos/prestamos.json
- Campos
    - prestamo_numero
    - prestamo
        - user_id
        - libro_id
        - regresado

7. Conclusiones

El desarrollo del Sistema de Gestión Bibliotecaria ha permitido cumplir exitosamente con los objetivos planteados, creando una solución funcional y eficiente para la administración de recursos bibliotecarios. A lo largo de este proyecto, el equipo logró integrar de manera efectiva conceptos teóricos aprendidos en la materia con su aplicación práctica en un sistema real.

7.1 Logros principales

Implementación completa de funcionalidades ABML: se desarrollaron exitosamente los módulos de Usuarios, Libros y Préstamos, permitiendo realizar operaciones de Alta, Baja, Modificación y Listado de manera robusta y eficiente. El sistema funciona con más de 1000 ejemplares de libros registrados manteniendo un desempeño ágil y funcional.

Aplicación efectiva de algoritmos de búsqueda: la implementación de búsqueda binaria por ISBN y búsqueda recursiva por texto libre demostró la importancia de utilizar algoritmos apropiados según el tipo de consulta. El sistema de índices ordenados permitió optimizar significativamente los tiempos de respuesta en las búsquedas más frecuentes.

Arquitectura escalable y mantenible: la decisión de utilizar una arquitectura en tres capas (presentación, negocio y data) facilitó la organización del código, la distribución del trabajo entre el equipo y la mantenibilidad del sistema. La separación de responsabilidades permitió que cada alumno pudiera trabajar en su módulo de forma relativamente independiente y después integrar su trabajo con el resto del sistema.

Uso efectivo de estructura de datos: La implementación demostró el uso práctico de listas, matrices, diccionarios y cadenas de texto.

7.2 Desafíos superados

Uno de los desafíos principales fue el diseño de la estructura de almacenamiento. La decisión de organizar los libros por género en carpetas separadas, en lugar de un único archivo JSON, requirió implementar funciones recursivas para recorrer directorios, pero resultó en un sistema más organizado y escalable.

La sincronización entre módulos, especialmente entre Libros y Préstamos, exigió un cuidadoso diseño para mantener la integridad de los datos. La actualización automática del estado de disponibilidad de los libros al registrar préstamos o devoluciones garantiza la consistencia en el sistema.

7.3 Conceptos académicos consolidados

Estructuras de datos: uso extensivo de listas, diccionarios y cadenas.

Algoritmos de búsqueda: búsqueda binaria y secuencial.

Modularización: diseño con funciones reutilizables con parámetros y valores de retorno claros.

Manejo de archivos: lectura y escritura de datos en formato JSON.

Validación de datos: control robusto de entradas del usuario.

Documentación: documentación robusta y profesional.

7.4 Oportunidades de mejora

Interfaz gráfica: la actual interfaz de consola es funcional pero limita significativamente la experiencia del usuario.

Autenticación: implementar roles y permisos permitiría un control más granular sobre las operaciones y actores del sistema.

Respaldos automáticos: nos protegería contra la posible pérdida de datos.

Notificaciones: alertas automáticas para devoluciones próximas o vencidas mejorarían la gestión de préstamos.

7.5 Reflexión Final

El desarrollo del sistema de Gestión Bibliotecaria representó una experiencia enriquecedora que permitió al equipo colaborar aplicando conocimientos teóricos en un contexto práctico.

El sistema resultante no solo cumple con los requisitos académicos sino que constituye una solución funcional que podría adaptarse para su uso en bibliotecas reales de pequeña a mediana escala.

Este proyecto sienta las bases para futuras mejoras y extensiones, demostrando que con una arquitectura sólida y un diseño bien pensado es posible crear soluciones escalables y mantenibles que resuelvan problemas reales del mundo bibliotecario.